

Breakout

Design

Nikola Panayotov

Table of Contents

Table of Contents	2
1. Introduction	3
2. UML Class Diagram	3
3. Screen-Flow Diagram.....	3
4. UI Wireframes.....	4
5. Data Model	4
6. Collision Design.....	4
7. New Functionality Beyond the Original Design	5
8. Source Code and Build Instructions.....	5

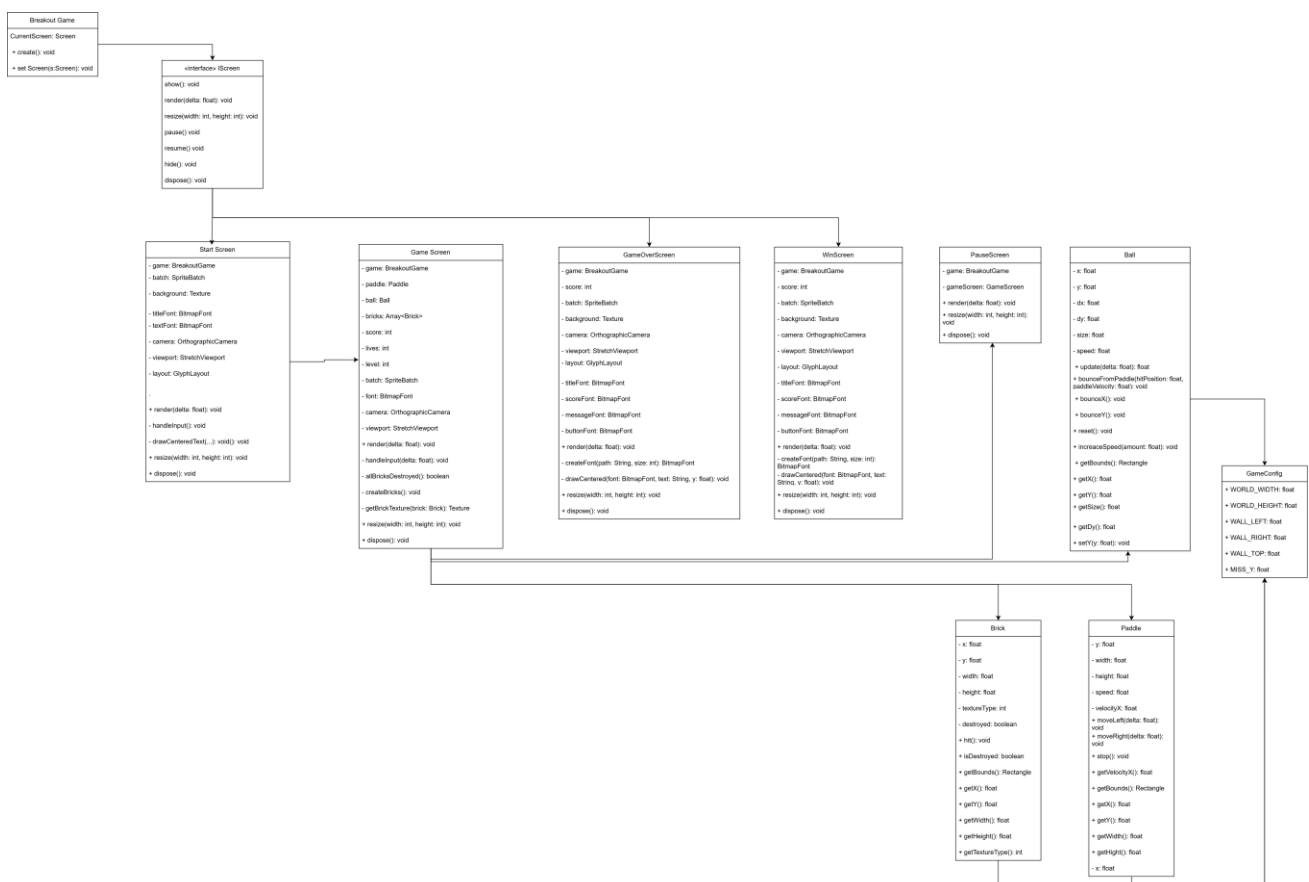
1. Introduction

This document contains the Design deliverables for the Breakout project. Design is the third of the five LO2 activities (Analyze, Advise, Design, Realize, Validate): it translates the requirements and technology choice from the Analysis & Advice document into the structure, behavior, and appearance of the actual game, so it can be built (Realize) and checked (Validate) against a clear plan.

Three design products are included: a UML class diagram (structure), a screen-flow diagram (behavior/navigation), and UI wireframes (appearance).

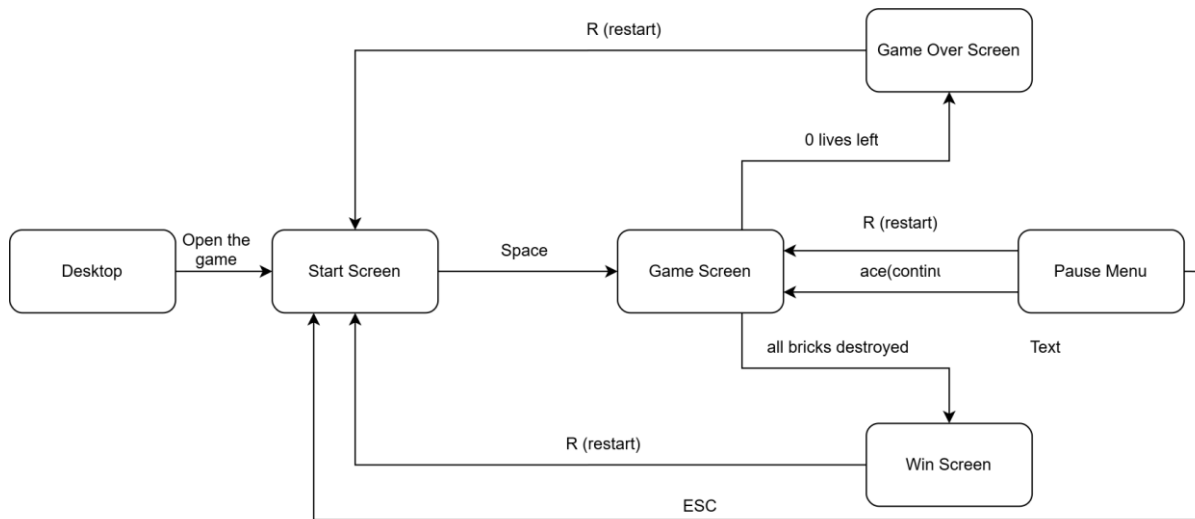
2. UML Class Diagram

The class diagram below shows the structure of the system: BreakoutGame holds the currently active IScreen; StartScreen, GameScreen, GameOverScreen, WinScreen, and PauseScreen all implement the IScreen interface; and GameScreen owns Paddle, Ball, and an array of Brick objects. Ball, Brick, and Paddle share a common GameConfig for shared constants such as world size and wall positions.



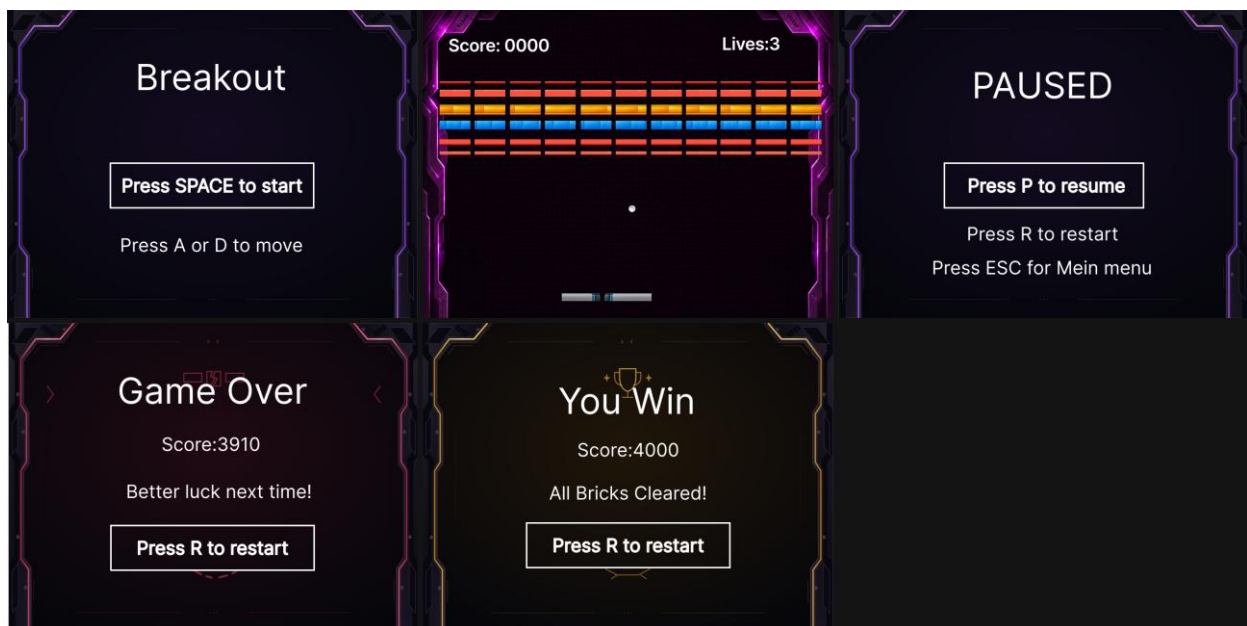
3. Screen-Flow Diagram

The diagram below shows how the game's screens connect, and what triggers each transition, including opening the game from the desktop and pausing/resuming mid-game.



4. UI Wireframes

The wireframes below show the Start, Game, Pause, Game Over, and Win screens, worked out before implementation to settle the layout of each screen (score/lives placement, button text, overall composition) before writing any rendering code.



5. Data Model

This project does not use a database or any persistent storage, so no Entity-Relationship Diagram (ERD) is included. The equivalent role — describing what data the system holds and how it's organized — is instead fulfilled by the class diagram in section 2: for example, GameScreen holds the score and lives as fields, and owns the list of Brick objects that make up the game's in-memory state.

6. Collision Design

Collision detection uses axis-aligned bounding boxes (AABB). Each game object (Paddle, Ball, Brick) exposes a `getBounds(): Rectangle` method, and GameScreen checks the ball's bounds against the paddle, each brick, and the screen edges every frame using `Rectangle.overlaps()`.

- Walls: overlapping the left/right edge inverts the ball's horizontal direction; overlapping the top edge inverts its vertical direction. Passing below the paddle costs a life instead.
- Bricks: on the first overlap with a non-destroyed brick, the brick is destroyed, the score increases, and the ball's vertical direction is reversed.
- Paddle: rather than a fixed bounce angle, the response blends where on the paddle the ball landed with the paddle's own velocity at the moment of contact, giving a "flick" effect (see requirement S01).

AABB was chosen over a full physics engine (e.g. LibGDX's Box2D) because all objects in this game are effectively rectangular, so overlap checks are fast and simple to reason about — a physics engine would add unnecessary setup for a game this size, without making the specific game-like bounce behavior above any easier to achieve.

7. New Functionality Beyond the Original Design

The following behavior exists in the realized game but was not part of the original design products in sections 2 to 6. It is documented here so the design stays consistent with what was actually built.

- Three levels with increasing difficulty (`GameScreen.createBricks()`): level 1 is a full grid, level 2 uses a checkerboard pattern, level 3 uses a pyramid pattern with more rows. Clearing all bricks advances to the next level and increases ball speed; only clearing level 3 shows the Win screen.
- Armored bricks: brick texture types 4 and 5 require two or three hits before being destroyed (`Brick.hit()`), rather than every brick breaking in one hit as described in section 6.
- A Level indicator is shown on the HUD alongside Score and Lives, extending the gameplay wireframe in section 4.
- Pressing ESC exits the game from the Start screen, and returns to the Start screen from the Pause screen; neither transition was part of the original screen-flow diagram in section 3.

8. Source Code and Build Instructions

The full source code for the design described above is available at <https://github.com/N1ki2K/breakout>. The project is built with Gradle and ships with the Gradle wrapper, so no separate Gradle installation is required. Run the commands below from the repository root:

- `gradlew.bat lwjgl3:run` on Windows, or `./gradlew lwjgl3:run` on Linux and macOS, launches the game directly.
- `gradlew.bat lwjgl3:jar`, or `./gradlew lwjgl3:jar`, builds a runnable JAR, which is written to `lwjgl3/build/libs`.